

Etch-a-Sketch

You're going to build an Etch-a-Sketch program in Python. You'll move a cursor around the screen with the WASD keys, drawing lines as you go. The program will be able to save your drawing to a file and load it back.

The `CHARS.py` module is provided. You'll need to `import` it, but you won't need to write it.

Part 1: Setup — Constants and CHARS Module

Create a new file called `draw.py` and add these lines:

```
import CHARS

CANVAS_WIDTH = 75
CANVAS_HEIGHT = 20

canvas = {}
cursorX = 0
cursorY = 0
```

The `CHARS` module provides box-drawing characters used to draw lines:

Testing

Temporarily add this test code at the bottom of your file to see the characters:

```
print(f"Vertical line: {CHARS.UP_DOWN}") // Should print |
print(f"Up-Right corner: {CHARS.UP_RIGHT}") // Should print ^
print(f"Cross: {CHARS.CROSS}") // Should print +
```

When you run the program, you should see the characters printed. Once you're satisfied, **remove the test code** before continuing.

Part 2: The Display Function

The canvas is a dictionary where each key is an `(x, y)` tuple (the column and row). The value at that key is a **set** that stores which directions have lines passing through that point — the letters 'W', 'A', 'S', and 'D'.

For example `canvas[(3, 5)]` might be `set(['W', 'S'])` meaning a vertical line passes through that cell.

The `getCanvasString` function builds a string that shows the entire canvas. Add this function after the `cursorY = 0` line:

```
def getCanvasString(canvasData, cx, cy):
    canvasStr = ''
    for rowNum in range(CANVAS_HEIGHT):
        for columnNum in range(CANVAS_WIDTH):
            if columnNum == cx and rowNum == cy:
                canvasStr += '#'
                continue

            cell = canvasData.get((columnNum, rowNum))
            if cell in (set(['W', 'S']), set(['W']), set(['S'])):
                canvasStr += CHARS.UP_DOWN
            elif cell in (set(['A', 'D']), set(['A']), set(['D'])):
                canvasStr += CHARS.LEFT_RIGHT
            elif cell == set(['S', 'D']):
                canvasStr += CHARS.DOWN_RIGHT
            elif cell == set(['A', 'S']):
                canvasStr += CHARS.DOWN_LEFT
            elif cell == set(['W', 'D']):
                canvasStr += CHARS.UP_RIGHT
            elif cell == set(['W', 'A']):
                canvasStr += CHARS.UP_LEFT
            elif cell == set(['W', 'S', 'D']):
                canvasStr += CHARS.UP_DOWN_RIGHT
            elif cell == set(['W', 'S', 'A']):
                canvasStr += CHARS.UP_DOWN_LEFT
            elif cell == set(['A', 'S', 'D']):
                canvasStr += CHARS.DOWN_LEFT_RIGHT
            elif cell == set(['W', 'A', 'D']):
                canvasStr += CHARS.UP_LEFT_RIGHT
            elif cell == set(['W', 'A', 'S', 'D']):
                canvasStr += CHARS.CROSS
            elif cell == None:
                canvasStr += ' '
        canvasStr += '\n'
    return canvasStr
```

Test it

Add this **temporary** test code at the **bottom** of your file to see if the function works:

```
# TEST: Create a small sample canvas with a corner at (1, 1)
testCanvas = {}
testCanvas[(1, 1)] = set(['S', 'D']) # right-down corner
testCanvas[(2, 1)] = set(['A', 'D']) # horizontal line
testCanvas[(1, 2)] = set(['W'])      # vertical line
print(getCanvasString(testCanvas, 3, 3))
```

Run the program. You should see a small drawing with a corner shape, and the cursor # at position (3, 3).

When you're satisfied, **remove the test code** before continuing.

Questions

1. What are the keys and values in the `canvas` dictionary? How do they represent the drawing?
2. How does the function know which character to draw at each position?
3. What does `canvasData.get((columnNum, rowNum))` return if that coordinate has no data?
4. What does the '#' character represent on the canvas?

Part 3: Movement — The make_move Function

When the player presses a WASD key, the cursor moves and the program records the direction at both the old and new positions. Add this function **after** `getCanvasString`:

```
def make_move(command):
    global canvas, cursorX, cursorY

    if canvas == {}:
        if command in ('W', 'S'):
            canvas[(cursorX, cursorY)] = set(['W', 'S'])
        elif command in ('A', 'D'):
            canvas[(cursorX, cursorY)] = set(['A', 'D'])

    if command == 'W' and cursorY > 0:
        canvas[(cursorX, cursorY)].add(command)
        cursorY = cursorY - 1
    elif command == 'S' and cursorY < CANVAS_HEIGHT - 1:
        canvas[(cursorX, cursorY)].add(command)
        cursorY = cursorY + 1
    elif command == 'A' and cursorX > 0:
        canvas[(cursorX, cursorY)].add(command)
        cursorX = cursorX - 1
    elif command == 'D' and cursorX < CANVAS_WIDTH - 1:
        canvas[(cursorX, cursorY)].add(command)
        cursorX = cursorX + 1
    else:
        return

    if (cursorX, cursorY) not in canvas:
        canvas[(cursorX, cursorY)] = set()

    if command == 'W':
        canvas[(cursorX, cursorY)].add('S')
    elif command == 'S':
        canvas[(cursorX, cursorY)].add('W')
    elif command == 'A':
        canvas[(cursorX, cursorY)].add('D')
    elif command == 'D':
        canvas[(cursorX, cursorY)].add('A')
```

Test it

Add this **temporary** test code at the **bottom** of the file:

```
# TEST: Move right twice, then down once, and display
make_move('D')
make_move('D')
make_move('S')
print("Canvas cells:", len(canvas))
print("Cursor at:", cursorX, cursorY)
print(getCanvasString(canvas, cursorX, cursorY))
```

Run the program. You should see:

- The cursor # at position (2, 1)
- A line drawn from (0, 0) to (2, 1)

When you're satisfied, **remove the test code** before continuing.

Questions

1. Why does the function use `global canvas, cursorX, cursorY`?
2. After moving right, why does the code add 'A' (left) at the new position?
3. What would happen if there were no bounds checks like `cursorY > 0`?

Part 4: The Main Game Loop

Now we'll make it interactive. Add this code **at the bottom** of the file:

```
moves = []

while True:
    print(getCanvasString(canvas, cursorX, cursorY))

    print('WASD keys to move, C to clear, '
          + 'F to save, or QUIT.')
    response = input('> ').upper()

    if response == 'QUIT':
        print('Thanks for playing!')
        break
    elif response == 'C':
        canvas = {}
        moves.append('C')
    elif response == 'F':
        try:
            filename = 'pic.txt'
            with open(filename, 'w', encoding='utf-8') as file:
                file.write(''.join(moves) + '\n')
                file.write(getCanvasString(canvas, None, None))
        except:
            print('ERROR: Could not save file.')

    for command in response:
        if command not in ('W', 'A', 'S', 'D'):
            continue
        moves.append(command)
        make_move(command)
```

Run the program. You should now be able to:

- Move the cursor with the **W, A, S, D** keys
- Clear the canvas with **C**
- Save your drawing with **F** (check the file `pic.txt`)
- Quit with **QUIT**

Try typing multiple keys at once like `WWDDDD` and pressing Enter.

Questions

1. Why is the input converted to uppercase with `.upper()`?

2. What does the `for command in response` loop do?

3. Why do we store moves in a list? What is each move used for?

Part 5: Loading Saved Drawings

When the program starts, it should load any saved drawing from `pic.txt`. Add this function **before** the `moves = []` line:

```
def load(filename):
    global canvas, cursorX, cursorY, moves

    try:
        with open(filename, 'r', encoding='utf-8') as f:
            moves_str = f.readline().strip()
    except:
        return

    if not moves_str:
        return

    canvas = {}
    cursorX = 0
    cursorY = 0

    for char in moves_str:
        if char == 'C':
            canvas = {}
            continue
        if char in ('W', 'A', 'S', 'D'):
            make_move(char)

    moves = list(moves_str)
```

Now **change** the line `moves = []` to load the saved file at startup:

```
moves = []
load('pic.txt')
```

Run the program again. If you saved a drawing earlier, it should appear when you start the program.

Questions

1. How does the `load` function reconstruct the canvas from the saved moves?

2. Why does the save file store both the moves string and the canvas display?

3. What happens if `load` is called when `pic.txt` doesn't exist?

Part 6: Experiments

Now that your Etch-a-Sketch works, try these experiments:

1. **Change the canvas size:** Modify `CANVAS_WIDTH` and `CANVAS_HEIGHT`. What happens when you make it very small? Very large?
2. **Change the cursor character:** Find where `'#'` is printed and change it to `'@'` or `'O'`.
3. **Draw then save and reload:** Draw something, save it with F, then restart the program. Does your drawing come back?
4. **Look at the save file:** Open `pic.txt` in a text editor. What's on the first line? What's on the remaining lines?
5. **Type multiple commands:** Try typing "WWDDDD" and pressing Enter. What happens?

Part 7: Questions

1. How does the `canvas` dictionary store the drawing data?
2. Why is a set used to store the directions at each point instead of a list?
3. In `make_move`, when the cursor moves from one cell to the next, why is an opposite direction added to the new cell?
4. What would happen if the cursor bounds checks were removed?
5. How does the `load` function reconstruct the drawing from the saved moves?
6. What does `CHARS.UP_DOWN_RIGHT` represent and when would it appear in a drawing?

Part 8: Challenges

Pick a challenge to extend your program:

Challenge 1: Show Cursor Position in the Prompt

Display the cursor's coordinates in the prompt so the player always knows where they are.

```
WASD keys to move, C to clear, F to save, or QUIT. Cursor: (12, 5)
```

Hint: The prompt is a `print()` statement. You already have `cursorX` and `cursorY` variables. Here's how to add them:

```
print('WASD keys... or QUIT. Cursor: ('  
      + str(cursorX) + ', ' + str(cursorY) + ')')
```

Or use an f-string:

```
print(f'WASD keys... or QUIT. Cursor: ({cursorX}, {cursorY})')
```

Challenge 2: Undo the Last Move

Add a **U** command that undoes the player's last move.

Hint: Look at how **load** rebuilds the canvas from scratch. Undo works the same way — remove the last move, reset everything, then replay all the remaining moves.

```
elif response == 'U':  
    if moves:  
        moves.pop()          # Remove the last move  
        canvas = {}  
        cursorX = 0  
        cursorY = 0  
        for char in moves:    # Replay all remaining moves  
            if char == 'C':  
                canvas = {}  
            elif char in ('W', 'A', 'S', 'D'):  
                make_move(char)
```

How does this work?

The moves list records every action. Removing the last one and rebuilding is like going back in time — the canvas is reconstructed as if that last move never happened.